Rapport de projet
Plateforme de petites annonces

BENYOUCEF Abderezak

Projet de langage web
Année universitaire 2025-2026

22 décembre 2025

Table des matières

1 Introduction **2**

2 Architecture de l'application **2**
2.1 Pattern MVCR (Modèle-Vue-Contrôleur-Routeur) 2
2.1.1 Pourquoi MVCR plutôt que MVC ? 2
2.1.2 Modèle 2
2.1.3 Vue 2
2.1.4 Contrôleur 3
2.1.5 Routeur 3
2.2 Pattern Stratégie pour le stockage 3
2.3 API client-serveur 3
2.3.1 Routes principales 4
2.3.2 Routes utilisateur 4
2.3.3 Routes administratives 4

3 Sécurité **4**
3.1 Protection CSRF (Cross-Site Request Forgery) 4
3.2 Validation des données 5
3.3 Gestion des permissions 5
3.4 Protection des fichiers uploadés 5
3.5 Sécurité des mots de passe 6
3.6 Protection contre les injections SQL 6

4 Aspects ergonomiques **6**
4.1 Navigation intuitive 6
4.2 Interface responsive 6
4.3 Gestion des images 6
4.4 Pagination 6

5 Difficultés rencontrées et solutions **6**
5.1 Architecture du stockage 6
5.2 Expérience utilisateur après connexion 7
5.3 Stockage des tableaux en SQL 7
5.4 Limite de taille des requêtes POST 7

6 Conclusion **8**

Introduction

Ce projet de plateforme de petites annonces a été développé dans le cadre du cours de langage web. L'objectif était de créer une application web complète permettant la mise en relation de particuliers pour la vente de biens d'occasion, en respectant des spécifications fonctionnelles précises tout en appliquant les bonnes pratiques de développement étudiées en cours.

L'application implémente les fonctionnalités essentielles d'une plateforme de petites annonces : dépôt d'annonces avec photos, recherche par catégories, système d'achat, gestion des utilisateurs et interface d'administration.

Architecture de l'application

Pattern MVCR (Modèle-Vue-Contrôleur-Routeur)

J'ai choisi d'implémenter une architecture MVCR qui étend le pattern MVC traditionnel avec un routeur dédié. Cette séparation des responsabilités permet une meilleure organisation du code et une maintenance facile

Pourquoi MVCR plutôt que MVC ?

Dans une architecture MVC classique, le contrôleur est responsable à la fois du routage (déterminer quelle action exécuter) et de la logique métier. Cette double responsabilité conduit rapidement à des contrôleurs monolithiques, complexes et difficiles à maintenir. Donc le pattern MVCR permet de séparer ces responsabilités : le routeur se charge de la navigation et de la résolution des actions, tandis que le contrôleur se concentre uniquement sur la logique applicative.

Modèle

- **Annonce** : Représente un bien mis en vente avec ses attributs (titre, description, prix, photos, etc.)
- **User** : Gère les informations des utilisateurs et leur rôle (utilisateur ou administrateur)
- **Category** : Structure les catégories de biens
- **Achat** : Enregistre les transactions entre utilisateurs

Chaque modèle est accompagné d'un **Builder** (AnnonceBuilder, UserBuilder, CategoryBuilder) qui valide les données avant création, respectant ainsi le principe de séparation des responsabilités. Les builders vérifient les contraintes (longueur des champs, format des données, etc.) avant la création des objets.

Vue

Le système de vue est organisé en trois couches :

- **View** : Classe principale qui orchestre l'affichage, construit le menu de navigation et gère les messages de feedback

- **UserView** : Gère les vues spécifiques aux utilisateurs (formulaires d'inscription, page de profil, détails d'annonce)
- **AdminView** : Gère l'interface d'administration avec des tableaux de gestion et des formulaires spécifiques

Cette séparation permet une spécialisation des vues selon les rôles et une meilleure organisation du code HTML généré.

Contrôleur

Trois contrôleurs spécialisés gèrent les différentes parties de l'application :

- **Controller** : Traite les actions principales liées aux annonces (consultation, création, suppression..) et aux achats
- **UserController** : Gère l'authentification (connexion, inscription, déconnexion) et le profil utilisateur
- **AdminController** : Contrôle les fonctionnalités réservé à l'admin (gestion des catégories, suppression d'utilisateurs)

Routeur

Le **Router** analyse les requêtes HTTP et les redirige vers les contrôleurs appropriés. Ses responsabilités incluent :

- L'analyse des paramètres GET et POST
- La redirection vers les méthodes appropriées des contrôleurs
- La génération d'URLs pour éviter les liens en dur dans les vues
- La gestion des redirections POST avec messages de feedback
- La validation CSRF pour toutes les requêtes POST

Pattern Stratégie pour le stockage

J'ai implémenté le pattern Strategy via des interfaces de stockage. Chaque entité possède une interface (ex : AnnonceStorage) avec deux implémentations :

- **AnnonceStorageSql** : Stockage dans une base de données MySQL avec PDO
- **AnnonceStorageSession** : Stockage temporaire dans la session PHP, utile pour les tests

cette architecture permet de changer le système de stockage sans modifier le reste du code. Il suffit de commenter/décommenter quelques lignes dans site.php

API client-serveur

L'API est basée sur une architecture REST-like avec des routes paramétrables. Le routeur génère dynamiquement les URLs, ce qui évite les liens en dur et facilite la maintenance.

Routes principales

- `GET /` : Page d'accueil avec catégories et dernières annonces
- `GET /?action=liste&category=ID` : Liste des annonces par catégorie avec pagination
- `GET /?id=ID_ANNONCE` : Détail d'une annonce
- ...

Routes utilisateur

- `GET /?action=connexion` : Formulaire de connexion
- `POST /?action=connexionPost` : Authentification
- `GET /?action=profil` : Interface personnelle
- ...

Routes administratives

- `GET /?action=admin` : Panneau d'administration
- `POST /?action=creerCategorie` : Création de catégorie
- `POST /?action=supprimerUtilisateur&email=EMAIL` : Suppression d'utilisateur
- ...

Sécurité

Plusieurs mesures ont été mises en place pour protéger l'application et ses utilisateurs :

Protection CSRF (Cross-Site Request Forgery)

J'ai implémenté une protection CSRF complète pour toutes les requêtes POST. La classe `TokenCSRF` gère cette fonctionnalité de manière centralisée :

```php
class TokenCSRF {
    public static function generate() {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        $token = bin2hex(random_bytes(32));
        $_SESSION['csrf_token'] = $token;
        $_SESSION['csrf_token_time'] = time();

        return $token;
    }

    public static function validate($token, $timeout = 3600) {
        if(session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        if (!isset($_SESSION['csrf_token'])) {
```

```
            return false;
        }

        if (!hash_equals($_SESSION['csrf_token'], $token)) {
            return false;
        }

        if(isset($_SESSION['csrf_token_time'])) {
            if((time() - $_SESSION['csrf_token_time']) > $timeout) {
                self::clear();
                return false;
            }
        }

        return true;
    }
}
```

Listing 1 – Classe TokenCSRF

Les tokens sont :

- Générés de manière cryptographiquement sûre avec `random_bytes()`
- Validés avec `hash_equals()` pour éviter les attaques par timing
- Expirés après 1 heure pour limiter la fenêtre d'attaque

Validation des données

Plusieurs couches de validation protègent l'application :

1. **Validation côté client** : Attributs HTML5 (`required`, `minlength`, `maxlength`, `type`)
2. **Validation côté serveur** : Filtrage et échappement des données avant stockage (validation dans les builders)

Gestion des permissions

Un système de rôles (user/admin) contrôle strictement l'accès aux fonctionnalités :

- Vérification systématique de l'authentification avant les actions sensibles
- Redirection vers la page de connexion avec préservation du contexte pour les actions nécessitant une authentification

Protection des fichiers uploadés

- Limitation à 5 photos maximum par annonce
- Validation du type MIME (JPEG uniquement) avec `finfo_file()`
- Taille maximale de 200 Ko par fichier
- Renommage aléatoire des fichiers avec `uniqid()` pour éviter les conflits et les attaques par injection de chemin
- Stockage dans un répertoire séparé (`uploads/annonces/`)

Sécurité des mots de passe

- Hashage avec `password_hash()` et vérification avec `password_verify()`
- Utilisation de l'algorithme bcrypt par défaut
- Minimum de 6 caractères exigé lors de l'inscription

Protection contre les injections SQL

- Toutes les requêtes SQL utilisent des requêtes préparées avec PDO.

Aspects ergonomiques

Navigation intuitive

- **Menu contextuel** : Le menu de navigation s'adapte dynamiquement au rôle de l'utilisateur (visiteur, utilisateur connecté, administrateur)
- **Feedback** : Messages de confirmation après chaque action

Interface responsive

Le CSS est conçu avec une approche mobile-first :

- Utilisation de flexbox et grid pour les dispositions complexes
- Media queries pour adapter l'affichage aux différentes tailles d'écran

Gestion des images

- **Vignette automatique** : La première photo sert automatiquement de vignette dans les listes
- **Galerie interactive** : Pour les annonces avec plusieurs photos, navigation avec miniatures cliquables
- **Fallback graphique** : Affichage d'un placeholder quand aucune photo n'est disponible

Pagination

La pagination est implémentée pour les listes d'annonces par catégorie :

- Limite de 10 annonces par page pour un chargement rapide
- Navigation complète (première, précédente, numéros de page, suivante)

Difficultés rencontrées et solutions

Architecture du stockage

Problème : Pendant le développement initial, le code était fortement couplé au système de stockage en session, ce qui aurait rendu difficile la migration vers une base de données.

Solution : J'ai refactorisé le code pour appliquer le pattern Stratégie. J'ai créé des interfaces communes (`AnnonceStorage`, `UserStorage`, etc.) et deux implémentations (Session et SQL). Cette approche m'a permis de développer rapidement avec le stockage Session, puis de migrer progressivement vers MySQL sans perturber le reste du code.

Expérience utilisateur après connexion

Problème : Quand un utilisateur non connecté tentait d'acheter un bien ou de déposer une annonce, il était redirigé vers la page de connexion. Après authentification, il se retrouvait sur la page d'accueil et devait retrouver manuellement l'annonce ou recommencer le dépôt.

Solution : J'ai implémenté un système de redirection post-login qui sauvegarde l'URL cible dans la session :

```php
// Dans le controleur, avant de rediriger vers la connexion
$_SESSION['redirect_after_login'] = $this->view->router->getAnnonceURL($id);
$this->view->router->POSTredirect($this->view->router->getLoginURL(),
                                  "Veuillez vous connecter pour acheter");

// Apres connexion reussie
$redirect = isset($_SESSION['redirect_after_login'])
            ? $_SESSION['redirect_after_login']
            : $this->view->router->getHomeURL();
unset($_SESSION['redirect_after_login']);
$this->view->router->POSTredirect($redirect, "connxion reussie");
```

Listing 2 – Système de redirection post-login

Stockage des tableaux en SQL

Problème : Les photos et les modes de livraison sont représentés sous forme de tableaux, or les bases de données SQL ne prennent pas en charge nativement ce type de structure.

Solution : Les tableaux sont sérialisés au format JSON à l'aide de la fonction `json_encode()` avant leur insertion en base de données, puis désérialisés avec `json_decode()` lors de la lecture.

Limite de taille des requêtes POST

Problème : Lorsqu'un utilisateur téléverse des photos trop volumineuses, PHP peut rejeter silencieusement la requête. Dans ce cas, la variable `$_POST` est vide, ce qui empêche toute validation côté serveur.

Solution : Une vérification de la taille de la requête est effectuée dans la méthode `AnnonceBuilder::isValid()` en comparant la valeur de `$_SERVER['CONTENT_LENGTH']` à une limite fixée à 8 Mo. Si cette limite est dépassée, un message d'erreur explicite est affiché à l'utilisateur.

Conclusion

Ce projet de plateforme de petites annonces m'a permis de mettre en pratique les concepts théoriques vus en cours. L'architecture MVC, combinée au pattern Stratégie pour le stockage, s'est révélée robuste, maintenable et extensible.

La gestion de la sécurité, notamment la protection CSRF et la validation des fichiers uploadés, a été intégrée efficacement. L'application respecte les spécifications fonctionnelles tout en offrant une expérience utilisateur fluide et intuitive.